

Basic (guess token counts, then check) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — word count × 1.3

There's no single "correct" set of 5 sentences — this is about the process, not a fixed answer. Here's a worked example table you can compare your own results against:

Sentence	Words	Guess (words × 1.3)	Actual tokens (check yourself)
"Hi."	1	~1-2	usually 1-2
"The cat sat on the mat."	6	~8	usually 6-8
"Unbelievably, the antidisestablishmentarianism debate continued."	4	~5	usually 9-11 — the guess is way off
"print('hello world')"	2 (code)	~3	usually 5-7 — code tokenizes differently than prose
"میں آج بازار جا رہا ہوں" (Urdu)	5	~7	usually much higher — non- English text uses more tokens per word

What this shows: the word-count guess works fine for short, plain, common English. It breaks down fast on long/uncommon words, code, and non-English text — exactly the pattern Doc03's Core Concepts warned about.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — chars ÷ 4 and words × 1.3, side by side

The same table, with both estimation methods shown side by side, and the actual mechanism explained for each surprising row:

Sentence	Char ÷ 4	Words × 1.3	Actual	Why the gap (if any)
"Hi."	~1	~1-2	1-2	No gap — trivially short, matches the rule directly.
"The cat sat on the mat."	~7	~8	6-8	Both estimates land close — this is the "average English sentence" the rule of thumb is calibrated for.
"Unbelievably, the antidisestablishmentarianism debate continued."	~13	~5	9-11	The word-count method fails badly here — one very long, rare word ("antidisestablishmentarianism") splits into many small sub-word tokens, but only counted as "1 word" in the word-based guess.
<code>print('hello world')</code>	~5	~3	5-7	Code has meaningful punctuation ((, ' ,)) that each often becomes its own token — word-counting code badly undercounts.
"میں آج بازار جا رہا ہوں" (Urdu)	varies	~7	often 2-3× higher	Non-Latin scripts were far less represented in training data used to build the token vocabulary, so the same <i>meaning</i> takes noticeably more tokens to express.

The actual mechanism, briefly: the tokenizer's vocabulary was built by finding the most common recurring chunks in a mostly-English, mostly-prose training set. Anything that looks like that training data (common English words) compresses efficiently into few tokens. Anything that doesn't (rare words, code syntax, other scripts) gets broken into smaller, less efficient pieces — because those exact chunks weren't common enough to earn their own single token.

Difference from Basic: Basic uses one estimate ($\text{words} \times 1.3$) and observes where it breaks. Intermediate runs two independent estimates side by side ($\text{chars} \div 4$ and $\text{words} \times 1.3$) across a deliberately wider range of sentence types, and explains the underlying mechanism (byte-pair-encoding-style vocabulary built mostly from English prose) that predicts *which kinds* of text will break the rule, not just that some of them do.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — the trailing-space test, and why it matters for cost, not just curiosity

Test strings:

```
A: "You are a helpful assistant. Answer concisely."
B: "You are a helpful assistant. Answer concisely. " (one trailing space added)
```

What actually happens, checked in the real tokenizer tool: string B does *not* just add one extra "space token" onto the end of string A's token sequence. Because tokenizers frequently merge a space with the token that follows it (a leading space is often baked into the next word's token, not treated as its own separate character), adding a trailing space at the very end — where there's no following word to merge with — typically does add a distinct final token, meaning A and B are *not* the same token sequence, even though a human reading both would call them identical.

Why this is the point, not a trivia fact: OpenAI's (and most providers') prompt caching matches on the literal token prefix. If your code builds a system prompt with `f"{instructions}\n\n"` in one code path and `f"{instructions}\n"` in another — or reads it from a file that a text editor silently added a trailing newline to — every call through the second path misses the cache silently. Nothing crashes, nothing errors, the response is still correct; the only symptom is a higher bill than expected, discovered much later, far from the line of code that caused it.

The habit this earns you: for any prompt text sent identically on every call (a system prompt, a fixed set of instructions), build it as one constant, defined once, never reconstructed slightly differently in two different places — and if you ever need to verify it's byte-identical across calls, compare the raw strings directly (or their `tiktoken` output) rather than eyeballing them.

Which estimate should you actually use, day to day? For a one-off "will this fit / roughly what will this cost" check, $\text{chars} \div 4$ in your head is enough — precision below about 20% doesn't change any real decision. For a system prompt or any other text resent unchanged on every call of a running feature, don't estimate at all: keep it as a single defined constant, and if caching behavior ever looks off, verify with `tiktoken` that what you're sending is actually byte-for-byte what you think it is, rather than trusting that two pieces of code that "look the same" produce the same tokens.